CS-GY 6643 Computer Vision – Project 1 Report

Reported model: Preserved `Baseline_091146`

1 Introduction and background

This project detects constellations by matching small query patches to a sky image. For each query, the output is either -1 if it is absent, or (x, y, m) if it is present. Here (x, y) is its centre in the original image and m indicates whether it belongs to the constellation figure. The program also chooses a constellation name from 48 supplied reference patterns.

The difficult part is that many stars look alike. A bright circular patch can match several places in the image, especially after blur, noise or changes in brightness. The reference patterns may also differ from the scene in size, orientation and shape. The method therefore combines local template matching with a check of the whole constellation's geometry.

The main ideas connect to the three course lectures: Gaussian filtering and correlation in Lecture 1 (pp. 105–128), detecting and matching informative points in Lecture 2 (pp. 52–55), and outlier handling and transformations in Lecture 3 (pp. 26–42 and 71–90). Page numbers refer to the supplied Fall 2026 PDF slides. The implementation adds ECC alignment, texture comparisons and one-to-one assignment; these extra techniques were not all covered in the slides.

The competition averages the following score equally across scenes:

$$S = 0.25P + 0.20L + 0.25G + 0.30I. \tag{1}$$

P is the macro-F1 for present/absent decisions, L measures query localization, G measures recovery of the figure's stars, and I is one for the correct constellation name. Localization earns full credit within 12 pixels, decreases linearly to zero at 36 pixels, and is zero for a missed query. Geometric recovery uses the same distance rule with one-to-one nearest-pair matching against all reported points. It does not directly measure the accuracy of the membership bits.

This report describes the saved `Baseline_091146` model, which received a reported public Kaggle score of **0.91146**. It uses classical image processing, with no trained neural network.

2 Datasets

2.1 Preprocessing

The supplied data contain three labelled scenes, 16 unlabelled scenes and 48 reference diagrams. Each sky image is 3000×3000 pixels and each query is 32×32. The labelled scenes contain 116 queries in total. The unlabelled scenes contain 668 queries, with 21–87 per scene.

Table 1: Label distribution in the three supplied labelled scenes. "Other present" means a real patch outside the constellation figure. Absent queries have no true scene coordinate.

Scene	Figure	Other present	Absent	Total
Pisces	10	17	14	41
Scorpius	10	16	15	41
Taurus	6	12	16	34
Total	26	45	45	116

There are more present queries than absent ones, and most present queries are outside the constellation figure. No oversampling or class balancing is used. All 851 PNGs in the saved dataset decoded successfully during the report check, with the expected scene and query dimensions.

The images are read in grayscale and converted to floating-point arrays. For the initial scene search, the code applies Gaussian smoothing with $\sigma = 1$, then subtracts a broad $\sigma = 7$ blur of that smoothed image. This reduces noise and slowly changing background, following the filtering ideas in Lecture 1. The result is resized to half resolution to reduce search time. Queries use the same background-subtraction step with initial smoothing $\sigma = 0.6$.

Candidate matches are then checked at full resolution. This step compares several blur levels, removes a local brightness plane, and compares high-pass and texture information. The texture calculation suppresses smooth radial star profiles so that surrounding detail contributes more to the match. A circular mask limits the comparison to the central part of the patch. Reflection padding handles scene borders.

For each reference diagram, the code extracts bright, opaque connected components of at least eight pixels and uses their centres as stars. These coordinates are centred and scaled by the largest coordinate span. The diagram's original pixel size is not treated as its expected size in the sky image.

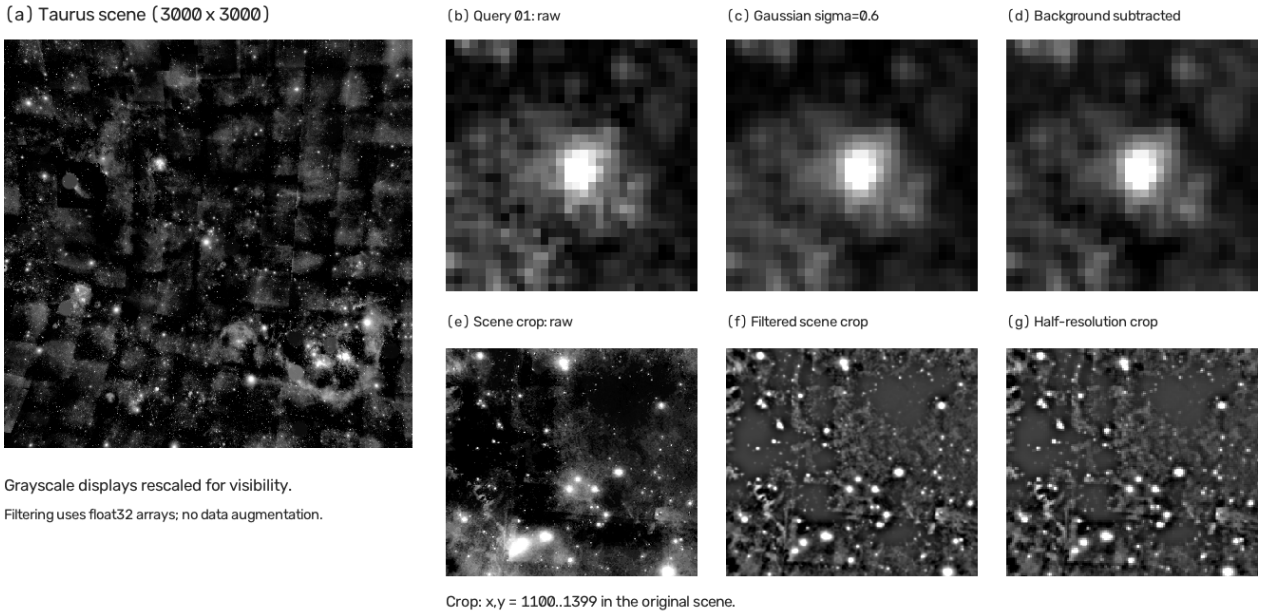


Figure 1: Taurus data and preprocessing. (a) Full scene. (b–d) A query before smoothing, after smoothing, and after background subtraction. (e–g) A scene crop before filtering, after filtering, and at half resolution. Panel contrast is adjusted for display only. The original-resolution images are retained for verification.

All three labelled scenes were used during development. There was no separate labelled holdout or data augmentation, so the local results should not be treated as independent validation. The unlabelled scenes are scored by Kaggle, with roughly 40% used for the public leaderboard. Missing or unreadable images cause an error, and nearly constant query patches return no candidates.

2.2 External and Auxiliary Datasets

No external data or pretrained weights are used. All images and reference patterns come from the competition. NumPy and OpenCV provide the numerical and image-processing operations.

3 Methods

The input is a sky image, its query patches and the reference patterns. The output contains query locations, presence and membership decisions, and a constellation name. There is no training loss. Instead, the method searches for strong image correlations and a consistent geometric arrangement.

3.1 Proposed Method

1. Find possible patch locations. The first stage extends the sliding correlation shown in Lecture 1. Each query is tested at three scales, 0.85, 1.00 and 1.15, and 24 rotations spaced 15° apart. Correlation

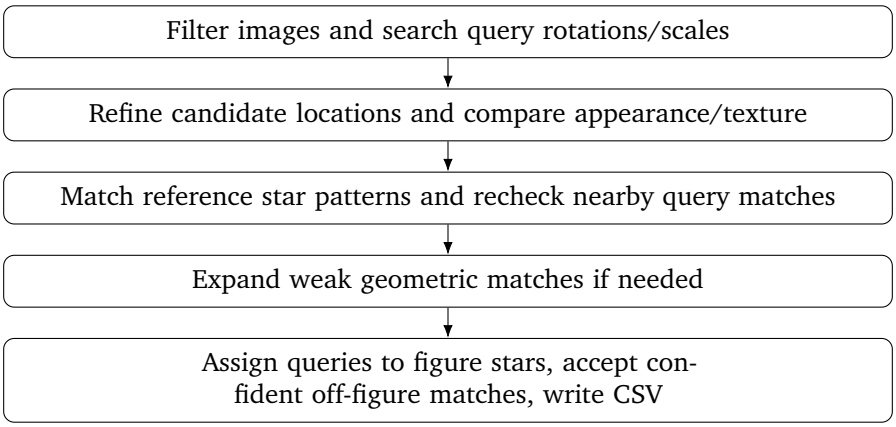


Figure 2: Main steps of the saved model. Local appearance proposes matches; constellation geometry helps resolve ambiguous ones.

is normalized by the local mean and variation so brightness alone does not dominate the score. Four peaks are collected per pose, then the best 48 spatially distinct positions are kept. Searching several poses addresses the scale and rotation issues discussed in Lecture 2.

2. Refine and verify the matches. Each candidate is refined using an affine transformation, $p' = Ap + t$, consistent with Lecture 3's treatment of coordinate transformations. The specific optimizer is OpenCV's enhanced correlation coefficient (ECC) alignment.[1] It runs for up to 60 iterations with tolerance 10^{-4}. Large or strongly distorted transforms are rejected.

The aligned image crop is checked at five blur widths: $0.4, 0.8, 1.2, 1.6, 2.0$. Its score combines brightness-corrected intensity correlation, high-pass correlation and texture correlation:

$$s = 0.2\rho_{\text{intensity}} + 0.2\rho_{\text{highpass}} + 0.6\rho_{\text{texture}}. \tag{2}$$

The best score over blur levels is retained. A separate appearance score averages intensity and high-pass correlation. Keeping both scores helps distinguish a genuinely similar patch from a generic bright spot. The candidate list keeps the union of the eight best score matches and eight best appearance matches, after spatial deduplication.

3. Check the constellation geometry. This stage follows Lecture 2's detect–describe–match structure. Bright star centres are detected in the scene, with a local-contrast detector added when faint stars are needed. Groups of four stars are described using normalized triangle areas and compared with the reference library through a FLANN index. Matching groups propose transformations of the reference figure into the scene.

The code tests those proposals against additional image stars and refines them with RANSAC-based homography fitting. RANSAC applies the outlier-handling idea from Lecture 3: some proposed correspondences will be wrong and should not determine the final fit. The four-star descriptors and projective fitting details are implementation additions to that course foundation. Projected figure nodes also guide another round of photographic checks, which can recover queries missed by the initial search.

4. Assign queries and recover weak figures. Queries compete for the projected figure nodes. Eligible matches must lie within 3.5 pixels of a node and pass score and appearance thresholds. Their weights reward strong photographic similarity and small geometric error. A Hungarian solver chooses a maximum-weight one-to-one assignment, allowing queries to remain unmatched. This extra step prevents several queries from being assigned to the same figure star. The winning constellation combines assignment weights with support from image stars.

If the best figure has image score below 4.5 or matches fewer than 75% of its stars, the code expands the search using faint stars and broader geometric hypotheses. Recovered models need extra query support because four points alone can fit an accidental homography. Balanced mode skips further projective

[1] OpenCV, `findTransformECC`: `https://docs.opencv.org/5.0/main_modules/video_track.html`.

searching when a broad affine model already has joint score at least 1.25, eight assigned queries and eight matched image stars. It still keeps the full photographic search.

5. Make the final decisions. Queries assigned to the winning figure receive $m = 1$. Other queries are accepted with $m = 0$ only when their best score is at least 0.50, appearance is at least 0.65, and the gap to the next candidate is at least 0.07. Remaining queries are marked absent. Coordinates are rounded in the original image, and the CSV follows the supplied row identifiers and patch counts. The constellation name is inferred from image evidence rather than from the scene filename.

4 Results

4.1 Training Details

There are no learned weights, epochs or optimizer. Development consisted of inspecting matches and adjusting rules on the three labelled scenes. The settings above are the final inference settings; a complete grid-search history was not recorded. OpenCV and sampled NumPy star combinations use seed 42. No repeated-seed or cross-validation experiment was performed.

The saved environment uses Windows, Python 3.14.5, NumPy 2.5.3 and OpenCV Python 5.0.0.93. Inference runs on the CPU, with one OpenCV thread per scene and several scenes processed concurrently. Fresh balanced-mode runs took 89.80 seconds for the three labelled scenes with three workers, and 556.98 seconds (about 9 minutes 17 seconds) for the 16 unlabelled scenes with six workers. Cache reuse was disabled. The logs do not record the processor model or RAM.

From the saved baseline folder, `runtime/python.exe -I -B run_model.py` regenerates the submission. The reported CSV is `code/results/submission_balanced.csv`. The saved folder contains its code, runtime and input data. Supporting checks and ablation results are included in `report_evidence.json` beside this report.

4.2 Results

Table 2: Local development results for the preserved model, evaluated on the three supplied labelled scenes. The final row averages scenes equally, matching the competition's aggregation rule.

Scene	Presence	Localization	Geometry	Identification	Score
Pisces	0.91717	0.92593	1.00000	1.00000	0.96448
Scorpius	0.89750	0.88462	1.00000	1.00000	0.95130
Taurus	0.70175	0.77778	1.00000	1.00000	0.88099
Mean	0.83881	0.86277	1.00000	1.00000	**0.93226**

The local mean is **0.93226**, compared with the reported public Kaggle score of **0.91146**. The public score is 0.02080 lower. Different scenes and repeated use of the development data could explain this gap, but the aggregate Kaggle score does not show which errors caused it. The private score and final rank are unavailable. No standard deviation across seeds or folds is reported because those experiments were not run.

All three local constellation names are correct, and geometric recovery is perfect. Individual query decisions still contain errors: there are nine false positives and eight missed present queries across the 116 queries. Taurus is the weakest scene, with six false positives and four misses. Also, recovering the correct set of figure stars does not guarantee that every query was assigned to its own correct star.

Decision-stage ablation. The saved candidates and geometric models were reused to test four final decision rules. The full replay reproduced the archived predictions exactly. Greedy assignment chooses the highest-weight available edges instead of solving the whole assignment. The other two variants suppress either figure-based query assignments or standalone query matches.

Optimal assignment improves the local score by 0.00513 over greedy assignment. The difference is in Scorpius, where two query outputs change and localization improves. Removing figure assignments

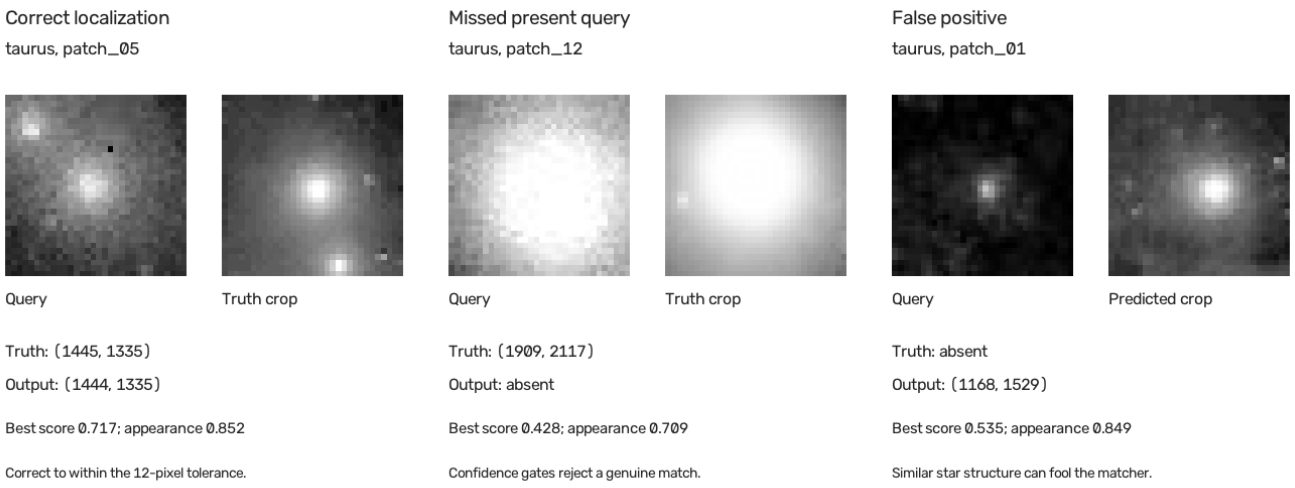


Figure 3: Three Taurus examples from the saved predictions. Left: a correct location, one pixel from ground truth. Middle: a real query rejected with best score 0.428. Right: an absent query accepted at a similar-looking star with score 0.553. Scene crops show the true location when available and the predicted location for the false positive; they have not been warped to match the query.

Table 3: Fixed-candidate decision ablations on the labelled development scenes. Geometry and candidate generation are held fixed; these are not fresh end-to-end pipeline runs or Kaggle scores.

Decision rule	Presence	Localiz.	Geometry	Identif.	Score
Full replay	0.83881	0.86277	1.00000	1.00000	**0.93226**
Greedy assignment	0.83881	0.83713	1.00000	1.00000	0.92713
No geometric assignments	0.76173	0.66857	0.46667	1.00000	0.74081
No standalone matches	0.56796	0.36277	1.00000	1.00000	0.76454

causes a much larger drop to 0.74081. Reporting only figure matches gives 0.76454, showing that finding off-figure queries also matters.

These are decision-stage tests, not complete pipeline reruns. Candidate generation, geometric rechecks and constellation identification are retained, including in the variant without figure assignments. The experiment therefore does not measure the separate contribution of ECC, texture scoring or faint-star recovery, and its scores are local rather than Kaggle results.

5 Use of AI

I used GPT 6 Astra and Codex for conceptual explanations, code generation and modification, debugging, score optimization, and report preparation. I supplied the assignment and lecture material, directed the experiments, submitted predictions to Kaggle, and reported the resulting scores. AI assistance also included figures, analysis scripts and report drafting.

6 Conclusions

The saved model reached a reported public score of 0.91146 by combining template matching with constellation geometry. Filtering and correlation provide candidate locations, while star arrangements help resolve ambiguous matches. The local decision tests show that both figure assignments and off-figure matches are needed; optimal assignment also improves query localization over greedy selection.

The main remaining problem is distinguishing similar-looking stars, particularly in Taurus. The method is also fairly slow, taking about nine minutes for the unlabelled batch. Only three labelled scenes were available and were reused during development, so the local score has limited value as a generalization estimate.

A useful next step would be to test each expensive component separately and spend extra search time only on uncertain queries or weak geometric matches. Any faster version should be compared with the saved baseline using both runtime and the individual score components, rather than only the public leaderboard score.